

PROJEKT DOKUMENTÁCIÓ

KLSZ Faloda

Online ételrendelő webalkalmazás

Készítették:

Pásztor Kevin
Kuti Levente Bánk
Kocsis Szabolcs Ernő

Szakma: Szoftverfejlesztő és -tesztelő
Tanév: 2025/2026

Tartalomjegyzék

1. Bevezetés.....	4
1.1 A projekt célja.....	4
1.2 Rövid leírás	4
1.3 Csapattagok és szerepkörök.....	4
2. Rendszer architektúra	5
2.1 Áttekintés	5
2.2 Technológiai stack	5
2.3 Rétegek leírása	5
2.3.1 Bemutató réteg (frontend)	5
2.3.2 Logikai réteg (backend)	6
2.3.3 Adatkezelési réteg (adatbázis)	6
2.4 Mappastruktúra.....	6
3. Adatbázis tervezés	7
3.1 Általános jellemzők.....	8
3.2 Táblák részletes leírása.....	8
3.2.1 users	8
3.2.2 restaurants	8
3.2.3 dishes.....	9
3.2.4 orders.....	9
3.2.5 order_items	9
3.2.6 payments.....	10
3.2.7 delivery.....	10
3.2.8 reviews.....	10
3.2.9 review_items	11
3.2.10 password_reset_tokens	11
3.3 Tárolt eljárások	11
3.3.1 Felhasználókezelés (users)	11
3.3.2 Étteremkezelés (restaurants, restaurant_hours)	12
3.3.3 Ételkezelés (dishes).....	12
3.3.4 Rendeléskezelés (orders, order_items).....	13
3.3.5 Fizetés és kiszállítás (payments, delivery).....	13
3.3.6 Értékelések (reviews)	14
3.3.7 Jelszó visszaállítás (password_reset_tokens)	14
4. Backend API	15
4.1 Áttekintés.....	15
4.2 Hitelesítés és jogosultságkezelés.....	15
4.3 REST végpontok részletesen	15
4.3.1 UsersController végpontjai	15
4.3.2 RestaurantsController végpontjai	16
4.3.3 DishesController végpontjai	16
4.3.4 OrdersController és OrderItemsController végpontjai	16
4.3.5 PaymentsController és DeliveryController végpontjai	17

4.3.6 ReviewsController végpontjai	17
4.3.7 PasswordResetTokenController végpontjai	17
4.4 Belső folyamatok	18
4.4.1 Háromrétegű kérelmfeldolgozás	18
4.4.2 CORS támogatás	18
4.4.3 Email értesítések	18
4.4.4 JPA és perzisztencia konfiguráció.....	18
5. Frontend.....	19
5.1 Áttekintés	19
5.2 Központi modul: api.js.....	19
5.3 Nyilvános oldalak	19
5.4 Étterem-specifikus oldalak.....	20
5.5 Felhasználói folyamat oldalak.....	20
5.6 Adminisztrációs oldalak	21
5.7 Reszponzív design.....	21
5.8 Kosár (LocalStorage alapú állapot)	22
6. Felhasználói folyamatok.....	23
6.1 Regisztráció	23
6.2 Bejelentkezés.....	23
6.3 Étterem böngészés és kosárba helyezés	23
6.4 Rendelés leadása és fizetés	23
6.5 Jelszó-visszaállítás.....	24
6.6 Profil módosítása és jelszócsere	24
6.7 Értékelések írása	24
6.8 Adminisztrátori funkciók	25
7. Tesztelés.....	26
7.1 Tesztelési stratégia.....	26
7.2 Backend tesztelés.....	26
7.3 Frontend tesztelés	26
7.4 Adatbázis tesztelés.....	26
8. Összefoglalás	28
8.1 Eredmények	28
8.2 Tanulságok	28
8.3 Lehetséges továbbfejlesztések.....	28
8.4 Záró gondolatok.....	29

1. Bevezetés

1.1 A projekt célja

A KLSZ Faloda egy online ételrendelő webalkalmazás, amely lehetővé teszi a felhasználók számára, hogy regisztráció és bejelentkezés után különböző pécsi és környékbeli éttermek kínálatából válogathassanak, ételeket helyezzenek a kosárba, megrendeljék a kiválasztott tételt, fizessenek értük, valamint utólag értékeljék a fogyasztott ételeket. Az alkalmazás egyszerre kínál egy klasszikus felhasználói (vendég) felületet és egy adminisztrációs felületet, amelyen keresztül a rendszergazda éttermeket, ételeket, felhasználókat, rendeléseket és fizetéseket tud kezelni.

A projekt elsődleges célja egy életszerű, valódi problémára megoldást nyújtó komplex szoftverrendszer megvalósítása volt, amely RESTful architektúrának megfelelő server- és kliensoldali komponenseket tartalmaz, többszintű felhasználói jogosultságkezelést biztosít, valamint a tiszta kód elveinek megfelelően íródott forráskóddal és teljes körű dokumentációval rendelkezik.

1.2 Rövid leírás

A KLSZ Faloda háromrétegű webalkalmazás. A frontend HTML5, CSS3, JavaScript és Bootstrap 5 segítségével készült, statikus oldalakat és dinamikus, AJAX-alapú kommunikációt egyaránt használ. A backend Java EE alapokon, JAX-RS REST API-val működik, WildFly alkalmazáserveren kerül futtatásra, és JWT alapú hitelesítést, BCrypt jelszóhash-elést, valamint JavaMail alapú email-küldést biztosít. Az adattárolást MySQL adatbázis végzi, amely 10 táblát és közel 80 tárolt eljárást tartalmaz. A backend és az adatbázis közötti kommunikáció kizárólag tárolt eljárásokon (CALL utasításokon) keresztül történik, ami egységes és karbantartható adathozzáférést biztosít.

Az alkalmazás támogatja a regisztrációt, bejelentkezést, profilkezelést, jelszó-visszaállítást email tokennel, étterem- és ételböngészést, kosárkezelést (LocalStorage alapon), rendelésleadást, kártyás-, PayPal és utánvétes fizetési módokat, értékelések írását és olvasását, valamint a teljes adminisztrációs munkafolyamatot.

1.3 Csapattagok és szerepkörök

A projekt háromfős fejlesztői csapatban valósult meg. A csapat tagjai együttműködve dolgoztak a feladatokon, Git verziókezelővel, GitHub repository-val és rendszeres egyeztetésekkel. A főbb felelőségek megosztása az alábbi táblázatban olvasható.

Név	Főbb felelősségi területek
Pásztor Kevin	Backend fejlesztés (JAX-RS controllerek), JWT hitelesítés, EmailService, integrációs tesztelés
Kuti Levente Bánk	Adatbázis-tervezés, MySQL séma, tárolt eljárás, adatbázis tesztelés
Kocsis Szabolcs Ernő	Frontend fejlesztés (HTML/CSS/JS oldalak, Bootstrap reszponzív design), admin dashboard, kosár- és fizetési folyamat

2. Rendszer architektúra

2.1 Áttekintés

A KLSZ Faloda klasszikus háromrétegű (three-tier) architektúra szerint épül fel. A bemutató (presentation) réteget a webböngészőben futó frontend alkalmazás, a logikai (business) réteget a WildFly alkalmazásszerveren futó Java EE backend, az adatkezelési (data) réteget pedig a MySQL adatbázis valósítja meg. A három réteg között HTTP/JSON, illetve JDBC kommunikáció zajlik, jól definiált interfészeken keresztül.

A frontend és a backend közötti adatcsere REST elveken alapul: a kliens HTTP kéréseket küld a /webresources URL alatt elérhető végpontokra, és JSON formátumú válaszokat kap. A felhasználó hitelesítését JWT (JSON Web Token) biztosítja, amely a bejelentkezés után a böngésző LocalStorage-ben kerül tárolásra, és a továbbiakban minden védett kérés Authorization fejlécében elküldésre kerül.

2.2 Technológiai stack

Az alábbi táblázat összefoglalja a projektben használt technológiákat és könyvtárakat.

Réteg	Technológia	Verzió	Szerep
Frontend	HTML5	-	Oldalstruktúra, szemantikus elemek
Frontend	CSS3	-	Stílusok, reszponzív design
Frontend	Bootstrap	5.3.2	UI keretrendszer, rács, komponensek
Frontend	JavaScript	-	Kliensoldali logika, fetch API
Backend	Java	23 (target 8)	Programozási nyelv
Backend	Java EE	7.0	Vállalati alkalmazás keretrendszer
Backend	JAX-RS	-	REST végpontok
Backend	WildFly	26.1.1	Alkalmazásszerver
Backend	JWT	0.11.5	JSON Web Token kezelés
Backend	jBCrypt	0.4	Jelszó hash-elés
Backend	JavaMail	1.6.2	Email küldés (SMTP)
Backend	org.json	20231013	JSON szerializáció
Adatbázis	MySQL	8.x	Relációs adatbázis
Build	Apache Maven	3.x	Függőségkezelés, build
Csomagolás	WAR	-	Webarchívum a WildFly-ra

2.3 Rétegek leírása

2.3.1 Bemutató réteg (frontend)

A bemutató réteg statikus HTML oldalakkól, CSS stíluslapokból és JavaScript modulokból áll. Az oldalak közötti navigációt a Bootstrap navbar biztosítja. Minden HTML oldal betölti a közös api.js

modult, amely központilag definiálja a backend végpontokat és a fetch alapú API hívásokat. A responzív viselkedés Bootstrap rács, valamint egyedi media queries és viewport-relatív mértékegységek (vw, vh, %, em, rem) használatával valósul meg. A frontend nem futtat saját szervert: bármilyen statikus tartalomszolgáltatóval (Apache, Nginx, vagy a WildFly statikus erőforrásként) kiszolgálható.

2.3.2 Logikai réteg (backend)

A backend Java EE alapú, EJB-vel és JAX-RS-szel. A package struktúra szerint elkülönül a controller, a service és a model réteg. A controller osztályok a HTTP végpontokat kezelik, validálnak és JSON választ építenek. A service osztályok @Stateless EJB-k, amelyek a tényleges üzleti logikát és az adatbázis-hívásokat tartalmazzák. A model osztályok JPA entitások, amelyek a táblákat reprezentálják, bár az adatbázis-műveleteket javarészt natív CALL utasítások végzik EntityManager.createNativeQuery() formában.

2.3.3 Adatkezelési réteg (adatbázis)

A MySQL adatbázis 10 táblát tartalmaz, amelyek a felhasználókat, éttermeket, ételeket, rendeléseket, fizetéseket, értékeléseket, kiszállításokat és jelszó-visszaállító tokeneket reprezentálják. A backend kód közvetlenül egyik táblát sem írja és olvassa, hanem kizárólag tárolt eljárásokon keresztül kommunikál. Ez biztosítja, hogy az adatbázis-séma változásai esetén csak az eljárások törzsét kell módosítani, a backend kód érintetlen marad. Az adatbázis kapcsolata a backenddel JDBC datasource-on keresztül, a WildFly-ban definiált java:/jdbc/etelrendelo JNDI néven keresztül történik.

2.4 Mappastruktúra

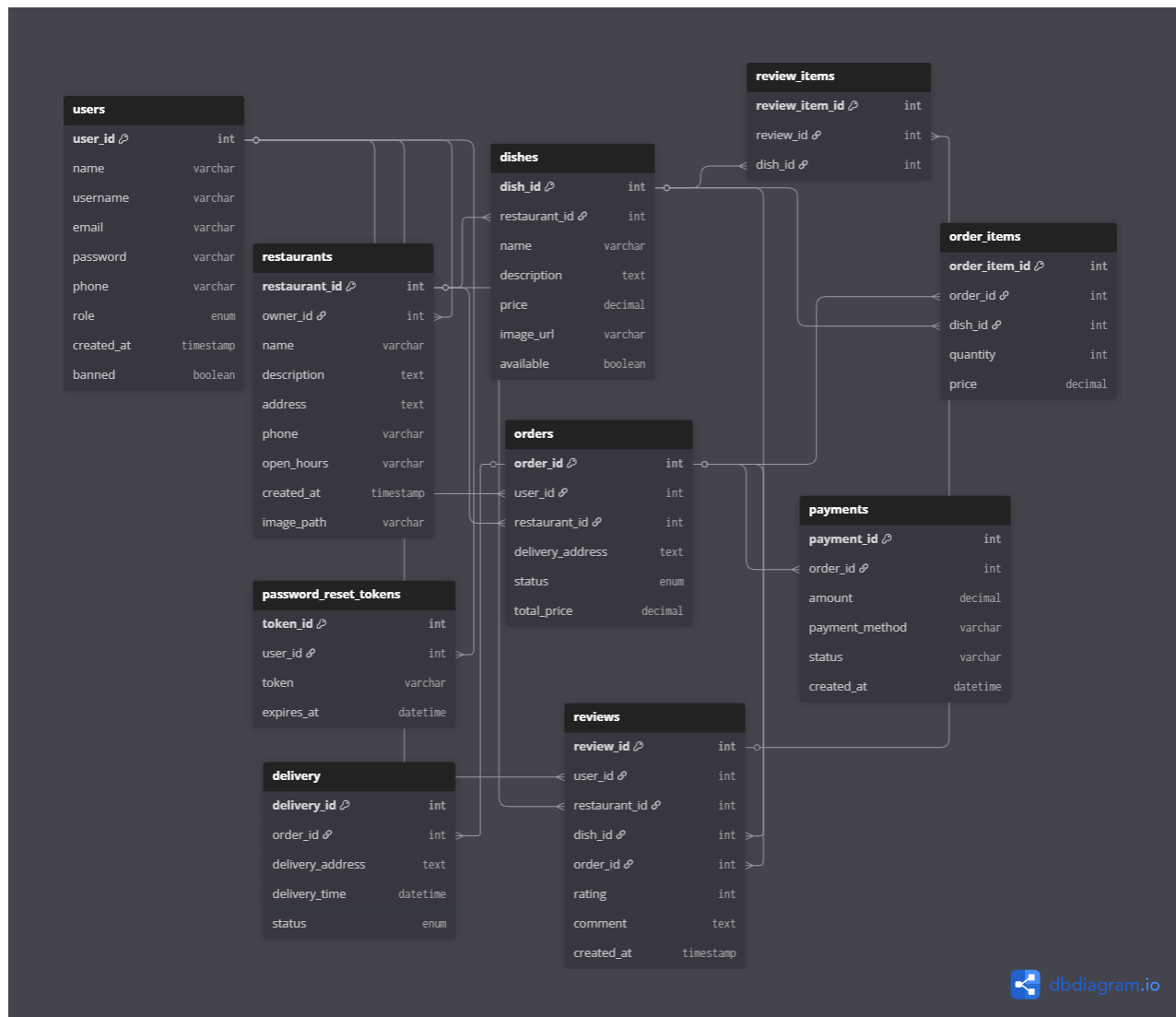
A teljes projekt három fő almappára tagolódik: Backend, Frontend és Adatbázis. Az alábbi struktúra mutatja a fontosabb mappák és fájlok elrendezését.

```
Final/
  Backend/
    vizsgaremek1/
      pom.xml
      src/
        main/
          java/com/mycompany/vizsgaremek1/
            config/          (CorsFilter, JWT)
            controller/      (10 db REST controller)
            model/            (JPA entitás osztályok)
            service/          (EJB szolgáltatások)
            resources/META-INF/persistence.xml
            webapp/WEB-INF/jboss-deployment-structure.xml
          target/vizsgaremek1.war
  Frontend/
    index.html, login.html, registration.html, profile.html
    cart.html, payment-final.html, restaurants.html
    admin-page.html, admin-dashboard.html, admin-users.html, ...
    api.js, login.js, registration.js, profile.js, cart.js, ...
    style.css, login.css, profile.css, cart.css, ...
  Adatbázis/
    etelrendelo1.sql
```

A backend Maven projektként van strukturálva, így a pom.xml egyetlen paranccsal képes lefordítani és WAR fájlba csomagolni a teljes alkalmazást. A frontend semmilyen build folyamatot nem igényel, közvetlenül HTML/CSS/JS forrásból fut. Az adatbázis dump (etelrendelo1.sql)

tartalmazza a teljes sémát, az összes tárolt eljárást, valamint a kezdeti adatokat (pécsi éttermek, ételek).

3. Adatbázis tervezés



3.1 Általános jellemzők

Az adatbázis neve etelrendelo1. Az adatbázis 10 táblát és 79 tárolt eljárást tartalmaz. Minden táblán külső kulcsok és indexek biztosítják az integritást és a gyors lekérdezést. A státusz típusú mezők ENUM típusúak, ami szerver oldalon korlátozza az érvényes értékek halmazát.

3.2 Táblák részletes leírása

3.2.1 users

A felhasználói fiókokat tárolja. Három szerepkört különböztet meg: customer (vásárló), admin (adminisztrátor), restaurant_owner (étteremtulajdonos). A jelszó BCrypt hash formában van tárolva. A banned mező 1-es értéke esetén a felhasználó nem tud bejelentkezni.

Mező	Típus	Megszorítás	Leírás
user_id	INT	PK, AUTO_INCREMENT	Felhasználó egyedi azonosítója
name	VARCHAR(255)	NULL	Teljes név
username	VARCHAR(255)	NULL, UNIQUE	Felhasználónév
email	VARCHAR(255)	NULL, UNIQUE	Email cím (bejelentkezéshez)
password	VARCHAR(255)	NULL	BCrypt hash-elt jelszó
phone	VARCHAR(50)	NULL	Telefonszám
role	ENUM	DEFAULT 'customer'	customer / admin / restaurant_owner
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Regisztráció időpontja
banned	TINYINT(1)	DEFAULT 0	Tiltott felhasználó jelölő

3.2.2 restaurants

Az éttermek alapadatait tárolja. Minden étteremnek van tulajdonosa (owner_id, idegen kulcs a users táblára). Az image_path mező a frontend statikus képeire mutató relatív útvonalat tartalmazza.

Mező	Típus	Megszorítás	Leírás
restaurant_id	INT	PK, AUTO_INCREMENT	Étterem egyedi azonosítója
owner_id	INT	NOT NULL, FK -> users	Tulajdonos felhasználó
name	VARCHAR(255)	NULL	Étterem neve
description	TEXT	NULL	Hosszabb leírás
address	TEXT	NULL	Cím

Mező	Típus	Megszorítás	Leírás
phone	VARCHAR(50)	NULL	Telefonszám
open_hours	VARCHAR(100)	NULL	Nyitvatartás szöveges formában
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Létrehozás időpontja
image_path	VARCHAR(255)	NULL	Borítókép útvonala

3.2.3 dishes

Az éttermek által kínált ételeket, italokat és desszerteket tárolja. A type mező szerint az adminisztrációs felület csoportosítva jeleníti meg az étlapot. Az available mező 0-ra állításával az étel ideiglenesen elrejthető a vendégek előtt.

Mező	Típus	Megszorítás	Leírás
dish_id	INT	PK, AUTO_INCREMENT	Étel egyedi azonosítója
restaurant_id	INT	NOT NULL, FK -> restaurants	Az étterem, ahol elérhető
name	VARCHAR(255)	NULL	Étel neve
description	TEXT	NULL	Leírás, összetevők
price	DECIMAL(10,2)	NULL	Ár forintban
image_url	VARCHAR(255)	NULL	Kép URL vagy útvonal
available	TINYINT(1)	DEFAULT 1	Elérhető-e (1) vagy sem (0)
type	ENUM	DEFAULT 'étel'	étel / ital / desszert

3.2.4 orders

A felhasználók által leadott rendeléseket tárolja. A status mező követi a rendelés életciklusát: pending (felvéve), preparing (készül), delivering (úton), completed (kézbesítve), cancelled (lemondva).

Mező	Típus	Megszorítás	Leírás
order_id	INT	PK, AUTO_INCREMENT	Rendelés egyedi azonosítója
user_id	INT	NOT NULL, FK -> users	Megrendelő
restaurant_id	INT	NOT NULL, FK -> restaurants	Az étterem
delivery_address	TEXT	NULL	Kiszállítási cím
status	ENUM	DEFAULT 'pending'	Rendelés státusza
total_price	DECIMAL(10,2)	NULL	Teljes összeg
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Leadás időpontja

3.2.5 order_items

Az egyes rendelési tételeket tartalmazza. Egy rendeléshez (order_id) több sor is tartozhat. Az ár (price) a rendelés pillanatában rögzítésre kerül, így későbbi árváltozások nem befolyásolják a már leadott rendeléseket.

Mező	Típus	Megszorítás	Leírás
order_item_id	INT	PK, AUTO_INCREMENT	Tétel egyedi azonosítója
order_id	INT	NOT NULL, FK -> orders	Melyik rendeléshez tartozik
dish_id	INT	NOT NULL, FK -> dishes	Melyik étel
quantity	INT	NULL	Darabszám
price	DECIMAL(10,2)	NULL	Egységár a rendelés pillanatában

3.2.6 payments

A rendelésekhez tartozó fizetéseket rögzíti. A method három értéket vehet fel: card (bankkártya), cash (utánvét), paypal. A status szintén állapotot követ: pending, paid, failed.

Mező	Típus	Megszorítás	Leírás
payment_id	INT	PK, AUTO_INCREMENT	Fizetés egyedi azonosítója
order_id	INT	NOT NULL, FK -> orders	A kapcsolódó rendelés
amount	DECIMAL(10,2)	NULL	Fizetett összeg
method	ENUM	NULL	card / cash / paypal
status	ENUM	DEFAULT 'pending'	pending / paid / failed
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fizetés időpontja

3.2.7 delivery

A kiszállítások állapotát követi nyomon. Egy rendeléshez egy kiszállítás tartozik. A status öt értéket vehet fel: pending (függőben), dispatched (átadva a futárnak), delivering (úton), completed (kézbesítve), failed (sikertelen).

Mező	Típus	Megszorítás	Leírás
delivery_id	INT	PK, AUTO_INCREMENT	Kiszállítás azonosítója
order_id	INT	NOT NULL, FK -> orders	A kiszállítandó rendelés
delivery_address	TEXT	NOT NULL	Pontos kiszállítási cím
delivery_time	DATETIME	NULL	Tervezett vagy tényleges idő
status	ENUM	DEFAULT 'pending'	Kiszállítás állapota

3.2.8 reviews

A felhasználói értékeléseket tárolja. Egy értékelés köthető egy étteremhez, egy konkrét ételhez, vagy egy adott rendeléshez (order_id). A rating egy 1-től 5-ig terjedő egész szám.

Mező	Típus	Megszorítás	Leírás
review_id	INT	PK, AUTO_INCREMENT	Értékelés azonosítója
user_id	INT	NOT NULL, FK -> users	Az értékelő felhasználó
restaurant_id	INT	NULL, FK -> restaurants	Az értékelt étterem
dish_id	INT	NULL, FK -> dishes	Az értékelt étel
order_id	INT	NULL, FK -> orders	A kapcsolódó rendelés
rating	TINYINT(4)	NOT NULL	1-5 csillagos osztályzat
comment	TEXT	NULL	Szöveges hozzászólás
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Értékelés időpontja

3.2.9 review_items

Egy értékelés több ételre is vonatkozhat. Ez a kapcsolótábla rögzíti, hogy egy review pontosan mely ételekre vonatkozik.

Mező	Típus	Megszorítás	Leírás
review_item_id	INT	PK, AUTO_INCREMENT	Sor azonosítója
review_id	INT	NOT NULL, FK -> reviews	Az értékelés
dish_id	INT	NOT NULL, FK -> dishes	Az étel

3.2.10 password_reset_tokens

Az elfelejtett jelszó visszaállításához használt egyszer használatos tokeneket tárolja. Az expires_at mező szerint a token érvényessége tipikusan 1 óra. A used mező 1-re állítása után a token többé nem használható.

Mező	Típus	Megszorítás	Leírás
token_id	INT	PK, AUTO_INCREMENT	Token azonosítója
user_id	INT	NOT NULL, FK -> users	Melyik felhasználónak
token	VARCHAR(255)	NOT NULL	Generált egyedi token
expires_at	DATETIME	NOT NULL	Lejárat dátuma és ideje
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Generálás időpontja
used	TINYINT(1)	DEFAULT 0	Felhasználták-e már

3.3 Tárolt eljárások

Az adatbázis-kommunikáció kizárólag tárolt eljárásokon keresztül zajlik. Ez biztosítja az adatbázis-műveletek központosítását, valamint védelmet nyújt az SQL injection ellen, mivel a backend mindig paraméterezett CALL utasításokat használ. Az alábbi táblázat témakörök szerint csoportosítva sorolja fel az összes 79 tárolt eljárást.

3.3.1 Felhasználókezelés (users)

Eljárás neve	Funkció
CreateUser	Új felhasználó beszúrása
GetAllUsers	Összes felhasználó listázása
GetUserById	Felhasználó lekérése azonosító alapján
UpdateUser	Felhasználó adatainak módosítása
DeleteUser	Felhasználó törlése
ChangeUserPassword	Jelszó módosítása régi ellenőrzésével
ForgotUserPassword	Jelszó beállítása token-alapú reset után
Login	Bejelentkezési lekérdezés email alapján
GetUsersByRole	Szerepkör szerinti szűrés
SearchUsersByEmail	Email alapú keresés
SearchUsersByName	Név alapú keresés
CountUsersByRole	Szerepkörönkénti darabszám
GetRecentUsers	Legutóbb regisztrált N felhasználó

3.3.2 Étteremkezelés (restaurants, restaurant_hours)

Eljárás neve	Funkció
GetAllRestaurants	Összes étterem listázása
AddRestaurant	Új étterem létrehozása
UpdateRestaurant	Étterem adatainak módosítása
DeleteRestaurant	Étterem törlése
SearchRestaurantsByAddress	Cím alapú keresés
GetTopRatedRestaurants	Legjobban értékelt N étterem
AddRestaurantHours	Nyitvatartás hozzáadása
UpdateRestaurantHours	Nyitvatartás módosítása
DeleteRestaurantHours	Nyitvatartási sor törlése
GetRestaurantHours	Étterem nyitvatartásának lekérése
ImportRestaurantHoursFromOpenHours	Migrációs eljárás a régi szöveges open_hours mezőből

3.3.3 Ételkezelés (dishes)

Eljárás neve	Funkció
GetAllDishes	Összes étel listázása
GetDishById	Egy étel adatainak lekérése

Eljárás neve	Funkció
GetDishByRestaurant	Egy étterem összes étele
GetAvailableDishesByRestaurant	Csak az elérhető ételek
AddDish	Új étel hozzáadása
UpdateDish	Étel adatainak módosítása
DeleteDish	Étel törlése
SearchDishesByName	Név alapú keresés
SearchDishesByPriceRange	Ársáv alapú szűrés
CountDishesByRestaurant	Éttermenkénti darabszám
CountDishesPerRestaurant	Csoportosított darabszám
GetCheapestDishByRestaurant	Legolcsóbb étel
GetMostExpensiveDishByRestaurant	Legdrágább étel
GetMostOrderedDishByRestaurant	Legtöbbet rendelt étel
GetTopRatedDishesByRestaurant	Legjobban értékelt N étel

3.3.4 Rendeléskezelés (orders, order_items)

Eljárás neve	Funkció
GetAllOrders	Összes rendelés listázása
GetOrderByld	Rendelés lekérése azonosító alapján
GetOrdersByUser	Felhasználó rendelései
AddOrder	Új rendelés létrehozása
UpdateOrder	Rendelés státusz vagy összeg módosítása
DeleteOrder	Rendelés törlése
GetActiveOrdersByRestaurant	Étterem aktív rendelései
GetTotalSpentByUser	Felhasználó összes költsége
GetAllOrderItems	Összes tétel listázása
GetOrderItemByld	Tétel lekérése
GetOrderItemsByOrder	Egy rendelés tételei
AddOrderItem	Tétel hozzáadása rendeléshez
UpdateOrderItem	Tétel módosítása
DeleteOrderItem	Tétel törlése

3.3.5 Fizetés és kiszállítás (payments, delivery)

Eljárás neve	Funkció
GetAllPayments	Összes fizetés listázása
GetPaymentById	Fizetés lekérése
GetPaymentByOrder	Egy rendeléshez tartozó fizetés
AddPayment	Új fizetés rögzítése
UpdatePayment	Fizetés státuszának módosítása
DeletePayment	Fizetés törlése
AddDelivery	Kiszállítás létrehozása
UpdateDeliveryStatus	Kiszállítás státuszának módosítása
DeleteDelivery	Kiszállítás törlése
GetDeliveryByOrder	Egy rendelés kiszállítása
GetPendingDeliveries	Függőben lévő kiszállítások

3.3.6 Értékelések (reviews)

Eljárás neve	Funkció
AddReview	Új értékelés rögzítése
GetReviewById	Értékelés lekérése azonosító alapján
GetReviewsByRestaurant	Étterem értékelései
GetReviewsByDish	Étel értékelései
GetReviewsByUser	Felhasználó értékelései
GetUserReviewsWithDetails	Felhasználó értékelései étterem és étel adatokkal
UpdateReview	Értékelés módosítása
DeleteReview	Értékelés törlése
GetAverageRatingByRestaurant	Étterem átlagos osztályzata
GetAverageRatingByDish	Étel átlagos osztályzata
GetRecentReviews	Legutóbbi N értékelés
SearchReviewsByComment	Kulcsszavas keresés a hozzászólásokban

3.3.7 Jelszó visszaállítás (password_reset_tokens)

Eljárás neve	Funkció
CreatePasswordResetToken	Új token generálása
GetPasswordResetToken	Token érvényességének lekérdezése
MarkPasswordResetTokenAsUsed	Token felhasználtra állítása

4. Backend API

4.1 Áttekintés

A backend tíz JAX-RS controller osztályból áll, amelyek mind a /webresources URL prefix alatt elérhetők. Az alábbi táblázat összefoglalja a controllereket és azok feladatát.

Controller	Bázis URL	Felelősség
UsersController	/users	Regisztráció, bejelentkezés, profilkezelés, jelszó
RestaurantsController	/restaurants	Éttermek listázása, létrehozása, módosítása
DishesController	/dishes	Ételek kezelése
OrdersController	/orders	Rendelések kezelése
OrderItemsController	/orderitems	Rendelési tételek kezelése
PaymentsController	/payments	Fizetések kezelése
DeliveryController	/delivery	Kiszállítások kezelése
ReviewsController	/reviews	Értékelések kezelése
PasswordResetTokenController	/password	Jelszó-visszaállítás email tokennel
EmailController	Email	Email-küldési segédfunkciók

4.2 Hitelesítés és jogosultságkezelés

A hitelesítést JWT (JSON Web Token) biztosítja, amely HMAC-SHA256 algoritmussal van aláírva. A token érvényességi ideje 24 óra. A token tartalmazza a felhasználó azonosítóját (subject), email címét, nevét és szerepkörét (role). A bejelentkezés után a frontend a token-t a böngésző LocalStorage-ben tárolja, és minden védett kérésnél az Authorization: Bearer <token> formátumban küldi el. A backend a JWT.validateTokenAndGetUserId() metódussal ellenőrzi a token érvényességét és kinyeri a felhasználó-azonosítót.

A jelszavakat a backend BCrypt algoritmussal hash-eli (jBCrypt 0.4 könyvtár). A regisztráció során egy salt-tal kombinált hash kerül az adatbázisba. A bejelentkezéskor a megadott jelszó a tárolt hash-sel kerül összehasonlításra a BCrypt.checkpw() függvénnyel. A jelszavak validációja az alábbi szabályok szerint történik:

- Minimum 8 karakter hosszúság
- Legalább egy nagybetű
- Legalább egy speciális karakter (!@#\$%^&* stb.)

A szerepkör-alapú hozzáférés-vezérlés a frontend oldalon is érvényesül: az admin oldalakat csak admin szerepkörrel rendelkező felhasználók érhetik el, a JWT role claim ellenőrzésével.

4.3 REST végpontok részletesen

4.3.1 UsersController végpontjai

HTTP	Útvonal	Funkció
GET	/users/GetAllUsers	Összes felhasználó listázása

HTTP	Útvonal	Funkció
GET	/users/GetUserById/{id}	Felhasználó adatainak lekérése
POST	/users/CreateUser	Új felhasználó regisztrálása + email
POST	/users/Login	Bejelentkezés, JWT visszaadása
PUT	/users/UpdateUser/{id}	Profil módosítása + email
DELETE	/users/DeleteUser/{id}	Felhasználó törlése
PUT	/users/ChangeUserPassword/{id}	Jelszó cseréje (rég ellenőrzéssel)

4.3.2 RestaurantsController végpontjai

HTTP	Útvonal	Funkció
GET	/restaurants/GetAllRestaurants	Összes étterem listázása
GET	/restaurants/GetRestaurantById/{id}	Egy étterem adatai
GET	/restaurants/SearchRestaurantsByAddress/{address}	Cím alapú keresés
POST	/restaurants/AddRestaurant	Új étterem létrehozása
PUT	/restaurants/UpdateRestaurant/{id}	Adatmódosítás
DELETE	/restaurants/DeleteRestaurant/{id}	Étterem törlése

4.3.3 DishesController végpontjai

HTTP	Útvonal	Funkció
GET	/dishes/GetAllDishes	Összes étel listázása
GET	/dishes/GetDishById/{id}	Egy étel adatai
GET	/dishes/GetDishesByRestaurant/{restaurantId}	Egy étterem ételei
POST	/dishes/AddDish	Új étel hozzáadása
PUT	/dishes/UpdateDish/{id}	Étel adatainak módosítása
DELETE	/dishes/DeleteDish/{id}	Étel törlése

4.3.4 OrdersController és OrderItemsController végpontjai

HTTP	Útvonal	Funkció
GET	/orders/GetAllOrders	Összes rendelés
GET	/orders/GetOrderById/{id}	Egy rendelés
GET	/orders/GetOrdersByUser/{userId}	Felhasználó rendelései
POST	/orders/AddOrder	Új rendelés létrehozása
PUT	/orders/UpdateOrder/{id}	Státusz módosítása
DELETE	/orders/DeleteOrder/{id}	Rendelés törlése

HTTP	Útvonal	Funkció
GET	/orderitems/GetAllOrderItems	Összes tétel
GET	/orderitems/GetOrderItemById/{id}	Egy tétel
GET	/orderitems/GetOrderItemsByOrder/{orderId}	Rendelés tételei
POST	/orderitems/AddOrderItem	Új tétel
PUT	/orderitems/UpdateOrderItem/{id}	Tétel módosítása
DELETE	/orderitems/DeleteOrderItem/{id}	Tétel törlése

4.3.5 PaymentsController és DeliveryController végpontjai

HTTP	Útvonal	Funkció
GET	/payments/GetAllPayments	Összes fizetés
GET	/payments/GetPaymentById/{id}	Egy fizetés adatai
GET	/payments/GetPaymentByOrder/{orderId}	Rendelés fizetése
POST	/payments/AddPayment	Fizetés rögzítése
PUT	/payments/UpdatePayment/{id}	Státusz módosítása
DELETE	/payments/DeletePayment/{id}	Fizetés törlése
GET	/delivery/GetDeliveryByOrder/{orderId}	Kiszállítás állapota
GET	/delivery/GetPendingDeliveries	Függőben lévő szállítások
POST	/delivery/AddDelivery	Új kiszállítás
PUT	/delivery/UpdateDeliveryStatus/{id}	Státusz módosítása
DELETE	/delivery/DeleteDelivery/{id}	Kiszállítás törlése

4.3.6 ReviewsController végpontjai

HTTP	Útvonal	Funkció
GET	/reviews/GetReviewById/{id}	Egy értékelés
GET	/reviews/GetReviewsByRestaurant/{restaurantId}	Étterem értékelései
GET	/reviews/GetReviewsByUser/{userId}	Felhasználó értékelései
GET	/reviews/GetAverageRating/{restaurantId}	Étterem átlagos osztályzata
GET	/reviews/GetRecentReviews/{limit}	Legutóbbi N értékelés
GET	/reviews/SearchByComment/{keyword}	Kulcsszavas keresés
POST	/reviews/AddReview	Új értékelés létrehozása
PUT	/reviews/UpdateReview/{id}	Értékelés módosítása
DELETE	/reviews/DeleteReview/{id}	Értékelés törlése

4.3.7 PasswordResetTokenController végpontjai

HTTP	Útvonal	Funkció
POST	/password/ForgotUserPassword	Token generálása + email küldése
POST	/password/ResetUserPassword	Új jelszó beállítása token alapján
GET	/password/ValidateToken/{token}	Token érvényességének ellenőrzése

4.4 Belső folyamatok

4.4.1 Háromrétegű kérelmfeldolgozás

Egy tipikus REST kérés feldolgozása három lépésben zajlik. Először a controller fogadja a kérést, validálja a paramétereket és a JSON body-t, majd hívja a megfelelő service metódust. A service @Stateless EJB, amely az EntityManager-en keresztül natív CALL utasítással meghívja a megfelelő tárolt eljárást. Az eljárás eredménye a service-ben kerül feldolgozásra (típuskonverzió, modell objektumok), majd a controller JSONObject vagy JSONArray formában építi fel a választ és visszaadja a kliensnek megfelelő HTTP státuszkóddal.

4.4.2 CORS támogatás

A backend külön CorsFilter osztályt tartalmaz, amely a Cross-Origin Resource Sharing fejléceket állítja be minden válasznál. Ez teszi lehetővé, hogy a frontend más eredetről (például localhost:5500) is el tudja érni a backendet. A filter beállítja az Access-Control-Allow-Origin, Access-Control-Allow-Methods és Access-Control-Allow-Headers fejléceket, valamint kezeli az OPTIONS preflight kéréseket.

4.4.3 Email értesítések

Az EmailService @Stateless EJB Gmail SMTP-n keresztül küld emaileket TLS titkosítással. Az aszinkron üzenetküldést a @Asynchronous annotáció biztosítja, így a felhasználó nem várakozik az email kézbesítésére. Az alábbi eseményekhez küld a rendszer automatikus emailt:

- Sikeres regisztráció üdvözlő emaillel
- Profil módosítása esetén értesítés a régi és új adatokkal
- Jelszó módosítás megerősítése
- Elfelejtett jelszó esetén jelszó-visszaállító link
- Új rendelés visszaigazolása összegzéssel

Az email-ek HTML formátumúak, közös sablonnal készülnek (KLSZ Faloda branding, sárga kiemelő szín, sötétkék háttér). A fejlécben az étterem logója és neve, a láblécben elérhetőségek és kapcsolati információk találhatók.

4.4.4 JPA és perzisztencia konfiguráció

A persistence.xml fájl egy etelrendelo nevű perzisztenciaegységet definiál, amely a java:/jdbc/etelrendelo JNDI datasource-on keresztül kapcsolódik a MySQL adatbázishoz. A datasource a WildFly konfigurációjában (standalone.xml) kerül létrehozásra, így környezeti változók nélkül, központilag kezelhető a kapcsolódás. A modellosztályok JPA entitások, de a tényleges adatbázis-műveletek tárolt eljárásokon keresztül történnek, ezért az automatikus séma-generálást nem használjuk.

5. Frontend

5.1 Áttekintés

A frontend statikus HTML, CSS és JavaScript fájlokból áll. A felhasználói felületet Bootstrap 5.3.2 keretrendszer egészíti ki, amelyet CDN-ről töltünk be. Az oldalak közötti navigációt a fix pozíciójú navbar biztosítja. A backend API-val való kommunikáció a fetch API-n keresztül, JSON formátumban történik. A bejelentkezett felhasználó adatai és a JWT token a böngésző LocalStorage-ben tárolódnak, így az oldal újratöltése után is megmarad a session.

A teljes frontend mintegy 25 HTML, 22 JavaScript és 16 CSS fájlból áll. A struktúra logikai csoportokra oszlik: nyilvános oldalak, felhasználói oldalak, étterem-specifikus oldalak és adminisztrációs oldalak. Az alábbi alfejezetek részletesen bemutatják az egyes csoportokat.

5.2 Központi modul: api.js

Az api.js fájl tartalmazza a teljes API-réteget. Egy központi konstansban (API_BASE_URL = 'http://localhost:8080/vizsgaremek1/webresources') tárolódik a backend bázis URL-je, így környezet váltása esetén egyetlen helyen kell módosítani. Egy API_ENDPOINTS objektum tartalmazza az összes végpontot kategorizálva (restaurants, users, dishes, orders, orderItems, payments, reviews). Az apiCall(url, method, data) általános függvény végzi a fetch hívásokat: beállítja a Content-Type fejléct, JWT token meglétekor automatikusan hozzáadja az Authorization fejléct, és egységesen kezeli a JSON szerializációt.

5.3 Nyilvános oldalak

A nyilvános oldalak bejelentkezés nélkül is elérhetők. Itt találhatók a marketing célú információs oldalak, valamint az autentikációs felületek.

Fájl	Funkció
index.html	Főoldal: hero szekció, népszerű étek, kategóriák, hírlevél
index.css	Főoldal egyedi stílusa (gradient, animációk)
index.js	Főoldal dinamikus elemei (legjobb étek betöltése)
index-logged-in.html	Bejelentkezett felhasználó főoldali változata
restaurants.html	Étek listázó oldal
restaurants.css	Étek-listázó stílusa
restaurants-loader.js	Étek dinamikus betöltése a backendből
ettermeink.html	Statikus éttermi áttekintés
login.html	Bejelentkezés űrlap
login.css	Bejelentkezés stílusa
login.js	Bejelentkezési logika, JWT mentése
registration.html	Regisztrációs űrlap
registration.css	Regisztráció stílusa
registration.js	Regisztrációs validáció és küldés

Fájl	Funkció
reset-password.html	Új jelszó beállítása token alapján
style.css	Globális közös stíluslap

5.4 Étterem-specifikus oldalak

Minden étteremnek van saját HTML oldala, amely az étterem brandingjét tükröző egyedi háttérképet, színvilágot és étlapot jelenít meg. A restaurant-sablon.html sablonként szolgál az új éttermek létrehozásához. Az étlap dinamikusan a backend GetDishesByRestaurant végpontjáról töltődik be.

Fájl	Funkció
restaurant-sablon.html	Általános étterem oldal sablon
restaurant-Pizza_Hut.html	Pizza Hut oldala
restaurant-csulokbar.html	Csülök Bár oldala
restaurant-Best_Food_Grill.html	Best Food Grill oldala
restaurant-reggeli.html	Reggeliző oldala
restaurant-szunetkave.html	Szünet Kávézó oldala
restaurant-Matyas-kiraly-vendeglo.html	Mátyás Király Vendéglő
restaurant-Teca_Mama_Vendeglo.html	Teca Mama Vendéglője
restaurant-elemozsia-Bistro.html	Elemózsia Bistro
restaurant-JuiceBoo_Pecs_Smoothie_Bar.html	JuiceBoo Pécs Smoothie Bar
restaurant.js	Étlap-betöltés, kosárba helyezés logikája
restaurant-scroll-cart.js	Görgéskor megjelenő mini-kosár
style_restaurants.css	Étterem oldalak közös stílusa
average-rating.js	Átlagos értékelés megjelenítése

5.5 Felhasználói folyamat oldalak

Ezek az oldalak a bejelentkezett felhasználó számára érhetők el. A profil, kosár, fizetési és értékelési funkciókat tartalmazzák.

Fájl	Funkció
profile.html	Felhasználói profil: adatok, rendelési előzmények, értékelések
profile.css	Profil oldal stílusai
profile.js	Profil-tabok kezelése, adatmódosítás, jelszócsere

Fájl	Funkció
cart.html	Kosár tartalmának megjelenítése
cart.css	Kosár stílusai
cart.js	Tételek listázása, mennyiség módosítása, törlés
payment-final.html	Fizetési felület (kártya, PayPal, utánvét)
payment-final.css	Fizetés stílusai
payment.js	Fizetési logika és AddOrder/AddPayment hívások
payment-card.js	Kártyaszám validáció (Luhn algoritmus)
reviews.html	Értékelések oldal
reviews.css	Értékelések stílusai
reviews.js	Csillag-értékelés interakció, hozzászólások
navbar-scroll.js	Navbar effektje görgetéskor

5.6 Adminisztrációs oldalak

Az adminisztrációs felület csak admin szerepkörrel érhető el. A frontend a JWT-ből kinyert role mező alapján engedélyezi vagy tiltja a hozzáférést, illetve az admin-auth.js minden admin oldal betöltésekor ellenőrzi a jogosultságot.

Fájl	Funkció
admin-page.html	Admin főoldal navigációval
admin-dashboard.html	Statisztikák és áttekintés
admin-dashboard.css	Dashboard stílusai
admin-dashboard.js	Statisztikák lekérése és megjelenítése
admin-users.html	Felhasználók kezelése
admin-users.css	Felhasználók kezelő stílusai
admin-users.js	CRUD műveletek felhasználókon
admin-orders.html	Rendelések kezelése
admin-orders.css	Rendelések stílusai
admin-orders.js	Rendelés-státuszok módosítása
admin-payments.html	Fizetések kezelése
admin-payments.css	Fizetések stílusai
admin-payments.js	Fizetés-státuszok módosítása
admin-auth.js	Admin jogosultság ellenőrzése minden admin oldalon

5.7 Reszponzív design

Az alkalmazás teljesen reszponzív, asztali, tablet és mobil eszközökön is jól használható. A reszponzivitást három fő technika biztosítja. Először, a Bootstrap 12-oszlopos rácsa: a col-12, col-md-6, col-lg-4 osztályokkal a tartalom automatikusan átrendeződik a képernyőméret függvényében. Másodszor, az egyedi CSS media queries: a 768px alatti szélességeknél a navbar összezsukódik hamburger menüvé, a kétoszlopos elrendezések egyoszlopos lineárisá alakulnak. Harmadszor, a viewport-relatív mértékegységek (vw, vh, %, em, rem) használata, amelyek szükségtelenné teszik a fix pixelértékek többségét.

A meta viewport tag minden oldal head szakaszában megtalálható: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`. A képek max-width: 100% szabályozással automatikusan a konténerhez igazodnak. A navbar fixed-top pozíciója lehetővé teszi, hogy a felhasználó bármilyen oldalon, bármikor elérje a fő navigációs elemeket.

5.8 Kosár (LocalStorage alapú állapot)

A kosár tartalma a böngésző LocalStorage-ben tárolódik, így a felhasználó akkor is megőrizheti a tételeket, ha bezárja a böngészőt. A cart kulcs alatt egy JSON tömb tárolódik, amely tételenként a következő mezőket tartalmazza: dish_id, name, price, quantity, restaurant_id. A kosár.js modul biztosítja a hozzáadás, módosítás, törlés és összegszámítás funkcióit. A fizetés véglegesítésekor a kosár tartalma a backend AddOrder és AddOrderItem hívásokkal kerül feltöltésre az adatbázisba, majd a LocalStorage-ből törlődik.

6. Felhasználói folyamatok

6.1 Regisztráció

Az új felhasználó a `registration.html` oldalon megadja a nevét, felhasználónevét, email címét, jelszavát (kétszer) és telefonszámát. A frontend először kliensoldalon validálja az adatokat: a két jelszónak egyeznie kell, az email cím formailag helyes legyen, a jelszó megfeleljen a komplexitási szabályoknak (minimum 8 karakter, legalább egy nagybetű, legalább egy speciális karakter).

A regisztráció elküldése a `POST /users/CreateUser` végpontra történik JSON formátumban. A backend újból validálja a beérkezett adatokat (server-side validation), majd ellenőrzi, hogy az email cím vagy a felhasználónév még nem foglalt-e (`emailExists`, `usernameExists` hívásokkal). Ha minden rendben, a `UserService.hashPassword()` BCrypt segítségével hash-eli a jelszót, és a `CreateUser` tárolt eljárás létrehozza a rekordot. Sikeres létrehozás után a backend automatikusan üdvözlő emailt küld a felhasználónak az `EmailService.sendRegistrationEmail()` metódussal, amely az aszinkron működés miatt nem blokkolja a választ.

A frontend a sikeres válasz (HTTP 201) után automatikusan átirányítja a felhasználót a `login.html` oldalra, ahol be tud jelentkezni. Hibás kérés esetén (409 Conflict, 400 Bad Request) az alkalmazás a backend hibaüzenetét megjeleníti az űrlap felett.

6.2 Bejelentkezés

A `login.html` oldalon a felhasználó megadja az email címét és jelszavát. A `login.js` a `POST /users/Login` végpontra küldi az adatokat. A backend a `Login` tárolt eljárással lekéri a megadott emailhez tartozó felhasználót, majd a `BCrypt.checkpw()` függvénnyel összehasonlítja a megadott jelszót a tárolt hash-sel. Ha a jelszó megfelelő, és a `banned` mező 0, a `JWT.createToken()` metódus létrehoz egy 24 órás érvényességű JWT token-t, amely tartalmazza a `user_id`-t (subject), email-t, nevet és role-t.

A backend a választ tartalmazza: `token`, `user_id`, `name`, `username`, `email`, `role`, `banned`. A frontend ezeket a `LocalStorage`-ben tárolja. A `role` értékétől függően az alkalmazás eldönti, hogy az `index.html` (vásárlóknak) vagy az `admin-page.html` (adminoknak) oldalra irányítja át a felhasználót. Tiltott felhasználó (`banned = 1`) esetén a backend 403 Forbidden választ ad, és a frontend hibaüzenettel jelzi, hogy a fiók le van tiltva.

6.3 Étterem böngészés és kosárba helyezés

A felhasználó az `index.html`-ről vagy a `restaurants.html`-ről választhat éttermet. Az étterem oldalán a `restaurant.js` dinamikusan betölti az étterem adatait (`GetRestaurantById`), ételeit (`GetDishesByRestaurant`), és átlagos értékelését (`GetAverageRating`). Az ételek típus szerint csoportosítva (étel, ital, desszert) kerülnek kirakásra Bootstrap kártyákban.

Egy étel kosárba helyezésekor a JavaScript ellenőrzi, hogy szerepel-e már a tétel a kosárban. Ha igen, a darabszámot növeli; ha nem, új tételt szúr be. A kosár a `LocalStorage`-ben kerül elmentésre, és a navbar mini-kosár ikonja valós időben mutatja a tételek számát. A felhasználó a `cart.html` oldalra navigálva részletesen megtekintheti a kosár tartalmát, módosíthatja a darabszámokat, vagy eltávolíthat tételeket.

6.4 Rendelés leadása és fizetés

A kosár véglegesítése a `payment-final.html` oldalon történik. A felhasználó megadja a kiszállítási címet, és kiválasztja a fizetési módot (kártya, PayPal, utánvét).

A megerősítés gombra kattintva a `payment.js` végrehajtja a következő hívássorozatot:

- POST /orders/AddOrder a rendelés alapadataival (user_id, restaurant_id, delivery_address, status='pending', total_price)
- A visszakapott order_id felhasználásával minden kosártételhez egy POST /orderitems/AddOrderItem hívás (order_id, dish_id, quantity, price)
- POST /payments/AddPayment a fizetés rögzítésére (order_id, amount, method, status='paid')
- POST /delivery/AddDelivery a kiszállítás létrehozásához (order_id, delivery_address, status='pending')

Sikeres rendelés után az EmailService visszaigazoló emailt küld a felhasználónak a megrendelt tételekkel és a teljes összeggel. A frontend a LocalStorage kosarat üríti, és átirányítja a felhasználót a profil oldalra a rendelési előzményekhez.

6.5 Jelszó-visszaállítás

Az elfelejtett jelszó folyamata két lépésben zajlik. Először a felhasználó a login oldalon a 'Elfelejtett jelszó' linkre kattint, majd megadja az email címét. A frontend POST /password/ForgotUserPassword kérést küld. A backend a PasswordResetTokenService:

- Ellenőrzi, hogy létezik-e a megadott email című felhasználó
- Generál egy egyedi, kriptográfiailag biztonságos tokent (UUID)
- A CreatePasswordResetToken eljárással elmenti az adatbázisba 1 órás érvényességgel
- Email-t küld a felhasználónak, amely tartalmazza a reset linket:
http://localhost:8080/reset-password.html?token=<token>

A felhasználó az emailben kapott linkre kattint, ami a reset-password.html oldalra navigál. Az oldal betöltésekor a JavaScript a GET /password/ValidateToken/{token} hívással ellenőrzi, hogy a token érvényes és nem járt le. Érvényes token esetén megjelenik az új jelszó űrlap. A felhasználó megadja és megerősíti az új jelszót, majd a POST /password/ResetUserPassword végpont a token alapján beállítja az új (BCrypt-tel hash-elt) jelszót, és a tokent felhasználtra állítja a MarkPasswordResetTokenAsUsed eljárással. A frontend egy megerősítő üzenettel és a login oldalra irányítással zárja a folyamatot.

6.6 Profil módosítása és jelszócsere

A profile.html három fő tabból áll: Adatok, Rendelési előzmények, Értékeléseim. Az Adatok tabon a felhasználó módosíthatja a nevét, felhasználónevét, email címét, telefonszámát és jelszavát. A módosítás PUT /users/UpdateUser/{id} kérésen keresztül történik. Ha az email vagy a felhasználónév megváltozik, a backend ellenőrzi, hogy az új érték nem foglalt-e. Sikeres módosítás után az EmailService egy összefoglaló emailt küld, amely táblázatos formában tartalmazza a régi és új adatokat.

A jelszó cseréjéhez a felhasználónak meg kell adnia a régi jelszavát is, ezzel ellenőrizve az identitását. A PUT /users/ChangeUserPassword/{id} végpont a régi jelszót BCrypt.checkpw()-vel hasonlítja össze, és csak egyezés esetén módosítja a jelszót. Sikeres művelet után megerősítő email kerül kiküldésre.

6.7 Értékelések írása

A felhasználó a kézbesített rendelései után értékelheti az ételt. Az értékelés egy 1-5 csillagos osztályzatot és egy opcionális szöveges hozzászólást tartalmaz. A reviews.html oldalon a csillagok kattintható ikonok, amelyek hover állapotban kiemelődnek. A POST

/reviews/AddReview végpont rögzíti az értékelést, a review_items kapcsolótábla pedig az érintett ételeket köti hozzá.

6.8 Adminisztrátori funkciók

Az admin felhasználó az admin-page.html-en éri el az adminisztrációs felületeket. Az admin-auth.js minden oldal betöltésekor ellenőrzi a JWT-ben tárolt szerepkört, és nem-admin felhasználót automatikusan átirányít a főoldalra. Az admin az alábbi területeken végezhet műveleteket:

Funkció	Leírás
Dashboard	Áttekintés: felhasználók száma, aktív rendelések, friss értékelések, top éttermek
Felhasználók	Listázás, keresés, létrehozás, szerkesztés, törlés
Rendelések	Aktuális rendelések követése, státuszváltás (pending → preparing → delivering → completed)
Fizetések	Fizetési státusz módosítása (pending → paid / failed)
Értékelések	Helytelen vagy spam értékelések törlése

7. Tesztelés

7.1 Tesztelési stratégia

A tesztelés három szinten történt: backend (API végpontok), frontend (felhasználói felületek) és adatbázis (tárolt eljárások és integritási megszorítások). A tesztelés részben manuálisan, részben Postman gyűjteményeket és böngésző fejlesztői eszközöket használva zajlott. A tesztek elsődleges célja a funkcionális helyesség ellenőrzése, valamint a hibakezelés és a határértékek tesztelése volt.

7.2 Backend tesztelés

A backend végpontok tesztelése Postman segítségével zajlott. Minden végponthoz pozitív (helyes adatokkal érkező) és negatív (hibás vagy hiányzó adatokkal érkező) teszteseteket készítettünk. A JWT-vel védett végpontok esetén különálló tesztek futtattunk hitelesítés nélkül, lejárt tokennel és érvényes tokennel. Az alábbi kép mutat a tesztekből részletet.

Task ID	Test Objective	Test Steps	Expected Result
75	74 CPG-51 Étterem törlése	DeleteRestaurant eljárás meghívása és étterem ID-ja megadása	200: Étterem sikeres törlése
76	75 CPG-51 nem létező étterem törlése	DeleteRestaurant eljárás meghívása és nem létező étterem ID-ja megadása	404: Étterem nem található
77	76 CPG-51 Étterem lekérdezése cím alapján	SearchRestaurantByAddress eljárás meghívása, majd egy cím megadása (pl: 7621 Pécs, Király utca 23-25.)	200: Sikeres lekérdezés
78	77 CPG-51 Étterem lekérdezése cím alapján (olyan cím ahol nincs étterem PL: Budapest)	SearchRestaurantByAddress eljárás meghívása, majd egy hibás cím megadása (pl: Budapest)	200: Sikeres lekérdezés (üres tömb)
79	78 CPG-51 Egy étel lekérdezése az ID-ja alapján	GetDishByID eljárás meghívása, majd egy étel ID-t beírni (pl: dishes/GetDishByID/99)	200: Sikeres lekérdezés
80	79 CPG-51 Egy étel lekérdezése nem létező ID alapján	GetDishByID eljárás meghívása, majd egy étel ID-t beírni (pl: dishes/GetDishByID/9999999)	404: Az étel nem található
81	80 CPG-51 Összes étel lekérdezése	GetAllDishes eljárás meghívása	200: Ételek sikeres lekérdezése
82	81 CPG-51 Ételek lekérdezése adott étteremből	GetDishesByRestaurant eljárás meghívása és egy étterem ID-ja megadása	200: Ételek sikeres lekérdezése
83	82 CPG-51 Ételek lekérdezése nem létező étteremből	GetDishesByRestaurant eljárás meghívása és egy nem létező étterem ID-ja megadása	200: Ételek sikeres lekérdezése (üres tömb)
84	83 CPG-51 Étél hozzáadása egy étteremhez	AddDish eljárás meghívása, majd az adatok beírása	200: Az étel sikeresen létrehozva
85	84 CPG-51 Étél hozzáadása egy étteremhez rossz étterem ID-val	AddDish eljárás meghívása, majd nem létező étterem ID-ját megadni	400: A megadott étterem nem létezik.
86	85 CPG-51 Étél frissítése	UpdateDish eljárás meghívása és a módosítandó étel ID-ját megadni.	200: Az étel sikeresen frissítve.
87	86 CPG-51 Étél frissítése nem létező ID-t megadni	UpdateDish eljárás meghívása és a nem létező ID-t megadni	404: Az étel nem található.
88	87 CPG-51 Étél törlése	DeleteDish eljárás meghívása és az étel ID beírásával (pl: dishes/DeleteDish/5)	200: Az étel sikeresen törölve
89	88 CPG-51 Nem létező étel törlése	DeleteDish eljárás meghívása és nem létező étel ID beírásával (pl: dishes/DeleteDish/999999)	404: Az étel nem található
90	89 CPG-51 Összes rendelés lekérdezése	GetAllOrders eljárás meghívása	200: Rendelések sikeres lekérdezése
91	90 CPG-51 Egy rendelés lekérdezése ID alapján	GetOrderByID eljárás meghívása és az adott Order ID-ját megadni	200: Rendelés sikeres lekérdezése
92	91 CPG-51 Egy rendelés lekérdezése nem létező ID alapján	GetOrderByID eljárás meghívása és nem létező Order ID-t megadni	404: A rendelés nem található
93	92 CPG-51 Rendelések lekérdezése userID alapján	GetOrderByUser eljárás meghívása és egy felhasználó ID-t megadni	200: Sikeres lekérdezés
94	93 CPG-51 Rendelések lekérdezése nem létező userID alapján	GetOrderByUser eljárás meghívása és egy nem létező felhasználó ID-t megadni	200: Ételek sikeres lekérdezése (üres tömb)
95	94 CPG-51 Rendelés leadása	AddOrder eljárás meghívása és adatok megadása	212: Rendelés sikeresen leadva
96	95 CPG-51 Rendelés leadása rossz felhasználó ID-val	AddOrder eljárás meghívása és rossz felhasználó ID megadása	404: A megadott felhasználó nem létezik
97	96 CPG-51 Rendelés frissítése	UpdateOrder eljárás meghívása és adatok beírása	200: A rendelés sikeresen frissítve
98	97 CPG-51 Rendelés frissítése rossz ID-val	UpdateOrder eljárás meghívása és rossz étel ID beírása	404: A rendelés nem található
99	98 CPG-51 Rendelés törlése	DeleteOrder eljárás meghívása és rendelés ID megadása	200: Rendelés sikeres törlése
100	99 CPG-51 Rendelés törlése rossz ID-val	DeleteOrder eljárás meghívása és rossz rendelés ID megadása	404: A rendelés nem található
101	100 CPG-51 Összes rendelési tétel lekérdezése	GetAllOrderItems eljárás meghívása	200: Összes tétel sikeres lekérdezése
102	101 CPG-51 Egy rendelési tétel lekérdezése ID alapján	GetOrderItemByID eljárás meghívása majd egy rendelési tétel ID megadása	200: Sikeres lekérdezés

7.3 Frontend tesztelés

A frontend tesztelése manuálisan zajlott. Az alábbi képen látható részlet az eredményekről.

158	CPG-40	Dashboard blocked (customer)	1. Login as customer	Access denied or redirected	Access denied or redirected	Pass	2026.02.20	Kocsis Szabolcs
159	CPG-40	Daily orders stat	2. Go to admin-dashboard	Today orders count displayed	Today orders count displayed	Pass	2026.02.20	Kocsis Szabolcs
160	CPG-40	Daily revenue stat	1. View dashboard	Daily revenue in Ft displayed	Daily revenue in Ft displayed	Pass	2026.02.20	Kocsis Szabolcs
161	CPG-40	Total revenue stat	1. View dashboard	Total revenue displayed	Total revenue displayed	Pass	2026.02.20	Kocsis Szabolcs
162	CPG-40	Total users stat	1. View dashboard	Total user count displayed	Total user count displayed	Pass	2026.02.20	Kocsis Szabolcs
163	CPG-40	Sidebar toggle	1. Click hamburger button	Sidebar toggles visibility	Sidebar toggles visibility	Pass	2026.02.20	Kocsis Szabolcs
164	CPG-40	Quick link - Orders	1. Click orders link	Navigate to admin-orders.html	Navigate to admin-orders.html	Pass	2026.02.20	Kocsis Szabolcs
165	CPG-40	Quick link - Users	1. Click users link	Navigate to admin-users.html	Navigate to admin-users.html	Pass	2026.02.20	Kocsis Szabolcs
166	CPG-40	Quick link - Payments	1. Click payments link	Navigate to admin-payments.html	Navigate to admin-payments.html	Pass	2026.02.20	Kocsis Szabolcs
167	CPG-40	Owner sections hidden	1. Login as admin (not owner)	Restaurant/dish sections hidden	Restaurant/dish sections hidden	Pass	2026.02.20	Kocsis Szabolcs
168	CPG-40	Owner sections visible	1. Login as owner	Restaurant and dish sections visible	Restaurant and dish sections visible	Pass	2026.02.20	Kocsis Szabolcs
169	CPG-40	Sidebar menu items	1. View sidebar	Dashboard, Orders, Users, Payments links	Dashboard, Orders, Users, Payments links	Pass	2026.02.20	Kocsis Szabolcs

7.4 Adatbázis tesztelés

Az adatbázis tesztelése MAMP-ban futó phpMyAdmin segítségével történt. A tárolt eljárásokat manuálisan, közvetlenül CALL utasításokkal hívtuk meg, és vizsgáltuk az eredményt. A tranzakciók helyességét, az integritási megszorításokat (idegen kulcs, NOT NULL, ENUM értékek) és a teljesítményt is teszteltük.

Tesztelt elem	Tesztelési mód	Várt eredmény	Eredmény
users tábla integritás	Duplikált email INSERT kísérlet	UNIQUE megsértés - hiba	OK
restaurants FK users-re	Nem létező owner_id	FK megsértés - hiba	OK

Tesztelt elem	Tesztelési mód	Várt eredmény	Eredmény
orders FK restaurants-ra	Nem létező restaurant_id	FK megsértés - hiba	OK
dishes type ENUM ellenőrzés	Érvénytelen érték (pl. 'snack')	ENUM hiba	OK
payments method ENUM	Érvénytelen érték	ENUM hiba	OK
CreateUser eljárás	Helyes paraméterekkel	Új sor + AUTO_INCREMENT	OK
Login eljárás	Létező email	User adat visszaadása	OK
Login eljárás	Nem létező email	Üres eredmény	OK
UpdateUser összes paraméterrel	Hash-elt jelszó frissítés	Sikeres UPDATE	OK
DeleteUser cascade	Felhasználó törlése	Kapcsolódó adatok kezelése	OK
AddOrder + AddOrderItem	Tranzakció szimuláció	Konzisztens beszúrás	OK
GetOrdersByUser	Több rendelésű user	Helyes lista	OK
GetTopRatedRestaurants	5-ös limit	Csillag szerinti rendezés	OK
GetAverageRatingByRestaurant	Étterem 12 értékeléssel	Helyes átlag (2 tizedes)	OK
SearchDishesByName	LIKE pattern '%pizza%'	Helyes találatok	OK
SearchDishesByPriceRange	1000-3000 közötti	Csak az ársávban	OK
CreatePasswordResetToken	Létező user	Token rögzítése	OK
GetPasswordResetToken lejárt	Tegnap expiring_at	Üres eredmény (érvénytelen)	OK
MarkPasswordResetTokenAsUsed	Felhasználás után	used=1 frissítés	OK
CountUsersByRole	3 customer, 1 admin	Helyes csoportosítás	OK
GetTotalSpentByUser	5 rendelésű user	Összeg helyes	OK
GetMostOrderedDishByRestaurant	GROUP BY teszt	Helyes top étel	OK
DECIMAL(10,2) precízió	12345.67 érték	Pontos kerekítés nélkül	OK

8. Összefoglalás

8.1 Eredmények

A KLSZ Faloda projekt eredményeként sikerült egy teljes körűen funkcionális, élesben is használható ételrendelő webalkalmazást kifejleszteni. Az alkalmazás megfelel a modern webfejlesztés legfontosabb elvárásainak: háromrétegű architektúra, RESTful API, JWT-alapú hitelesítés, BCrypt jelszó-hash-elés, reszponzív felhasználói felület, automatizált email-kommunikáció. A teljes rendszer 10 adatbázis-táblát, 79 tárolt eljárást, 10 REST controllert, 25 HTML oldalt, 22 JavaScript fájlt és 16 CSS stíluslapot tartalmaz.

A projekt során a csapat háromfős fejlesztői egységként, Git verziókezelő rendszer használatával, agile-szerű iterációkban dolgozott. Minden csapattag önállóan dolgozott a rá osztott területeken (backend, adatbázis, frontend), miközben rendszeres egyeztetésekkel és code review-val biztosította a részeredmények integrációját.

8.2 Tanulságok

A projekt megvalósítása során számos szakmai tapasztalatot szereztünk. A háromrétegű architektúra szigorú szétválasztása kezdetben több munkát jelentett, de utólag jelentősen megkönnyítette a hibakeresést és a funkcionalitás bővítését. A tárolt eljárásokon keresztüli adatbázis-elérés központosította az adatkezelést és védelmet nyújtott az SQL injection ellen. A JWT-alapú hitelesítés egyszerűbbnek bizonyult, mint a session-alapú megoldás, mivel az alkalmazás állapotalan (stateless) maradhat.

A frontend fejlesztés során a Bootstrap keretrendszer használata felgyorsította a reszponzív UI megvalósítását. A vanilla JavaScript (keretrendszer nélküli) megközelítés azt eredményezte, hogy a kód egyszerű és átlátható maradt, ugyanakkor a kód-újrafelhasználáshoz szükségünk lett egy közös api.js modulra. Egy nagyobb projektben mindenképpen érdemes lenne React vagy Vue keretrendszert használni a komponens-alapú fejlesztéshez.

8.3 Lehetséges továbbfejlesztések

A jelenlegi alkalmazás már teljes mértékben működőképes, de számos területen továbbfejleszthető:

- Natív mobil alkalmazás (Android / iOS) készítése a vásárlói élmény javítására
- Valós fizetési szolgáltató (Stripe, Barion) integrációja a demonstrációs fizetés helyett
- Térképes étterem-kereső Google Maps API használatával
- Push értesítések a rendelés állapotáról
- Kuponok és kedvezmények rendszere
- Több nyelvű felület (angol, német) i18n támogatással
- Frontend keretrendszer-alapú újraírás (React) komponens-alapú architektúrával
- HTTPS kötelező használata éles üzemben, Let's Encrypt tanúsítvánnyal
- Konténerizáció Docker segítségével a könnyebb telepítéshez
- CI/CD pipeline (GitHub Actions) automatizált build-hez és deployment-hez
- Részletesebb logging és monitoring (ELK stack, Prometheus)
- Adminisztrációs dashboard bővítése értékesítési statisztikákkal és diagrammokkal

8.4 Záró gondolatok

A KLSZ Faloda projekt sikeresen demonstrálja, hogy egy háromfős diákcsoport is képes egy komplex, valós problémára megoldást nyújtó webalkalmazást szakszerűen, modern technológiákkal megvalósítani. A projekt során szerzett tapasztalatok - a Java EE backend fejlesztéstől a responszív frontend tervezésen át az adatbázis-tervezésig - mind hozzájárultak a csapat szakmai fejlődéséhez. Az alkalmazás kódbázisa tiszta, jól strukturált és karbantartható, így alapot szolgáltathat akár egy későbbi tényleges termékesülési munkához is.

A projekt elkészítésében részt vett mindhárom csapattag: Pásztor Kevin, Kuti Levente Bánk és Kocsis Szabolcs Ernő. Köszönjük a felkészítő tanárok és konzulensek támogatását, akik a projekt teljes ideje alatt segítették a munkát és az iránymutatásukkal hozzájárultak a végeredmény minőségéhez.